Basic (your first coroutine) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `coroutine_basics_practice.py`: "`async def` doesn't run like a normal function" is the single fact every later exercise in this document assumes you already internalized — see it fail once, on purpose, here, before it costs you a confusing bug in real code. **If not:** the first time you'd meet an unawaited coroutine would be a silent `RuntimeWarning` buried in real output, with no smaller example to recognize it from.

Basic Version

Approach 1 — the direct way

```
# coroutine_basics_practice.py
import asyncio

async def wait_and_return():
    await asyncio.sleep(1)
    return "done"

# the mistake: calling it with no await and no asyncio.run
unstarted = wait_and_return()
print(unstarted)   # something like: <coroutine object wait_and_return at 0x...>

# the fix: asyncio.run actually starts the event loop and runs it to completion
result = asyncio.run(wait_and_return())
print(result)       # "done"
```

Expected output:

```
<coroutine object wait_and_return at 0x7f2a1c0b1a40>
done
sys:1: RuntimeWarning: coroutine 'wait_and_return' was never awaited
```

The last line is Python's own warning, printed when the program ends: the coroutine in `unstarted` was built but never run — the exact silent mistake this exercise is about. The first `print` shows a coroutine object — `wait_and_return()` on its own never ran the function body, it just built a

suspended coroutine and handed it back unstarted. The second `print` shows `"done"`, because `asyncio.run(...)` is what actually drives that coroutine to completion (waiting the 1 second along the way) and gives you its return value.

This version works correctly. It's missing type hints and the `if __name__ == "__main__":` guard — both fine for a first working version.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints and a `__main__` guard

```
# coroutine_basics_practice.py
import asyncio


async def wait_and_return() -> str:
    await asyncio.sleep(1)
    return "done"


if __name__ == "__main__":
    unstarted = wait_and_return()
    print(unstarted)   # <coroutine object wait_and_return at 0x...>

    result: str = asyncio.run(wait_and_return())
    print(result)       # "done"
```

Expected output:

```
<coroutine object wait_and_return at 0x7f2a1c0b1a40>
done
sys:1: RuntimeWarning: coroutine 'wait_and_return' was never awaited
```

Difference from Basic: the `-> str` type hint on `wait_and_return` documents, without reading the body, what the coroutine resolves to once it's awaited. The `if __name__ == "__main__":` guard means this file can be safely imported elsewhere (a test, another module) without the demonstration code running on import — only running when the file is executed directly. Same underlying behavior either way: calling an `async def` function builds a coroutine object, and only `await` or `asyncio.run(...)` actually runs it.

Approach 2 — a timeout, so a hang can't freeze the program forever

Story: a coroutine waiting on something outside your program — a network call, in every later document — can just hang forever with nothing here to stop it. **If not:** one slow or dead server would freeze your entire program indefinitely, with no way to recover short of killing the process.

```
# coroutine_basics_practice.py
import asyncio


async def wait_and_return() -> str:
    await asyncio.sleep(1)
    return "done"


async def main() -> None:
    try:
        result = await asyncio.wait_for(wait_and_return(), timeout=5)
        print(result)
    except asyncio.TimeoutError:
        print("wait_and_return took too long, giving up")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
done
```

And with `timeout=0.5` instead (shorter than the 1-second sleep inside `wait_and_return`):

```
wait_and_return took too long, giving up
```

`asyncio.wait_for(coro, timeout=...)` cancels the coroutine and raises `asyncio.TimeoutError` if it hasn't finished in time, instead of waiting on it forever. Notice `asyncio.run()` is called exactly once now, wrapping `main()` — every other coroutine in this file is reached with a plain `await`, never its own nested `asyncio.run()` call.

Approach 3 — safe to call whether or not a loop is already running

Story: `asyncio.run()` assumes it's the only thing starting an event loop — call it from inside a FastAPI route, a notebook, or any other `async def` function and it raises a confusing `RuntimeError` that points at asyncio internals, not at the actual mistake. **If not:** reusing this function from inside an already-async caller (exactly what `async_client_conversion` does) would surface as a cryptic crash instead of a clear message pointing at the fix.

```
# coroutine_basics_practice.py
import asyncio
```

```
async def wait_and_return() -> str:
    await asyncio.sleep(1)
    return "done"


def run_wait_and_return() -> str:
    """Run wait_and_return() from plain, synchronous code.

    Only call this from a script's true entry point. If you're already
    inside async code (a FastAPI route, another coroutine), just
    `await wait_and_return()` directly instead of calling this at all.
    """
    try:
        asyncio.get_running_loop()
    except RuntimeError:
        # no loop running -- safe to start one
        return asyncio.run(wait_and_return())
    raise RuntimeError(
        "run_wait_and_return() was called from inside a running event loop; "
        "await wait_and_return() directly instead"
    )


if __name__ == "__main__":
    print(run_wait_and_return())
```

Expected output:

```
done
```

This version fails loudly and clearly, with a message that actually explains what to do instead, if someone reuses `run_wait_and_return()` from inside code that already has an event loop running — instead of surfacing asyncio's own less obvious `RuntimeError: asyncio.run() cannot be called from a running event loop`.

Approach 4 — `asyncio.create_task`: start it now, collect the result later

Story: `await` waits right there, on that line. Sometimes you want a coroutine to start running *now* while your code does something else, and pick up its result later — like starting a slow lookup, then answering a quick question while it runs. **If not:** Doc19's "run a shadow comparison in the background" (`asyncio.create_task(...)`) would be the first time you ever started a task without waiting on it.

`asyncio.create_task(coro)` wraps a coroutine in a **task** and schedules it straight away. It starts running at your next `await`. Keep the task in a variable and `await` it later to get its result — a task nobody keeps or awaits can finish (or fail) without you ever seeing it.

```
# coroutine_basics_practice.py
import asyncio
import time


async def wait_and_return(label: str, seconds: float) -> str:
    await asyncio.sleep(seconds)
    return label + " done"


async def main() -> None:
    start = time.perf_counter()
    task = asyncio.create_task(wait_and_return("background", 2))
    print("task created, main() keeps going")

    foreground = await wait_and_return("foreground", 1)
    print(f"{foreground} after {time.perf_counter() - start:.1f}s")

    background = await task
    print(f"{background} after {time.perf_counter() - start:.1f}s")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
task created, main() keeps going
foreground done after 1.0s
background done after 2.0s
```

The whole run takes 2 seconds, not 3: the background task was already sleeping while `main()` waited on the foreground one. Compare with Basic's mistake — a plain `wait_and_return()` call builds a coroutine that never runs; `create_task()` builds a task that runs on its own.

Difference from Approach 1, and between Approaches 2–4: Approach 1's version trusts `wait_and_return()` to always finish, and trusts that `asyncio.run()` is always safe to call from wherever this code ends up. Approach 2 fixes the first assumption — a coroutine waiting on something outside your program (a network call, in every later document) needs a timeout, or a single hang can freeze everything indefinitely. Approach 3 fixes the second assumption — it turns a confusing `RuntimeError` from deep inside `asyncio` into a clear, specific message pointing at the

actual fix, for the specific case of this function being reused from inside an already-async caller. Approach 4 is a different tool: `create_task()` starts a coroutine running in the background, and a later `await` collects its result.

Which one should you actually write? For a throwaway script, Approach 1 is genuinely enough. The moment this kind of function is going to be imported and reused elsewhere in the curriculum — which is exactly what happens from `async_client_conversion` onward — Approach 2's timeout is worth adding on almost anything that waits on the outside world, since "it just hangs forever" is a much worse failure than "it raised a clear timeout error after 5 seconds." Approach 3's running-loop check is worth adding specifically to small entry-point helpers you expect other files to import — not to every coroutine, just the ones meant to be a program's front door. Use Approach 4's `create_task()` when work should start now but its result is only needed later — and always keep the task and `await` it.